

Operational Data Foundation Framework for SME Smart Manufacturing: Field Implementation and Evidence

Yoonseok Kang ¹  and Dongchul Park ^{2,*} 

¹ Department of Convergence Security, Chung-Ang University, Seoul 06974, South Korea; kangcl1609@cau.ac.kr

² Department of Industrial Security, Chung-Ang University, Seoul 06974, South Korea; dongchul@cau.ac.kr

* Correspondence: dongchul@cau.ac.kr

Abstract

Smart manufacturing increasingly depends on continuous and usable operational data, yet in constrained small and medium-sized enterprise (SME) factories, the practical bottleneck often lies not in downstream analytics or AI applications but in the absence of an operationally viable data foundation. This study addresses that problem by proposing an *Operational Data Foundation Framework* for constrained SME manufacturing environments. The framework is derived from a lifecycle-based problem analysis and formalizes six core requirements: continuity, governance, diagnosability, operability, reprocessability, and evolvability. These requirements are further translated into design principles and operational invariants, thereby providing a structured basis for both design and assessment. To examine its field relevance, the framework was instantiated in a real SME deployment involving heterogeneous environmental sensors, a retrofitted legacy scale, LoRa-based field communication, edge-side persistence, cloud-side stream processing, and a governed canonical event model. The evaluation was conducted from the perspective of operational viability rather than benchmark performance alone. Requirement-based evidence showed that the implemented system preserved controlled semantics, supported segmented observability, maintained replayable historical records, and remained manageable under constrained deployment conditions. A representative field case further demonstrated the framework's practical value: LoRa reception instability was diagnosed through segment-wise discrepancy analysis and improved from approximately 33% to 95% packet reception after targeted intervention. The study contributes a field-grounded design logic for operational data systems that can support monitoring, traceability, analytics, alerting, and future extensions in digitally constrained manufacturing settings.

Keywords: smart manufacturing; SME digitalization; industrial data infrastructure; data governance; edge–cloud architecture

1. Introduction

Smart manufacturing is transforming manufacturing from a fixed and fragmented production environment into a connected operational system in which sensing, communication, computation, and decision making are tightly intertwined. Technologies such as the Internet of Things (IoT), cyber–physical systems (CPS), artificial intelligence and machine learning (AI/ML), real-time monitoring, predictive maintenance, and digital twins exemplify this shift. In this context, manufacturing systems should therefore be understood not only as physical production arrangements, but also as data-mediated operational systems.

Received:

Revised:

Accepted:

Published:

Copyright: © 2026 by the authors.

Submitted to *Systems* for possible open access publication under the terms and conditions of the [Creative Commons Attribution \(CC BY\)](https://creativecommons.org/licenses/by/4.0/) license.

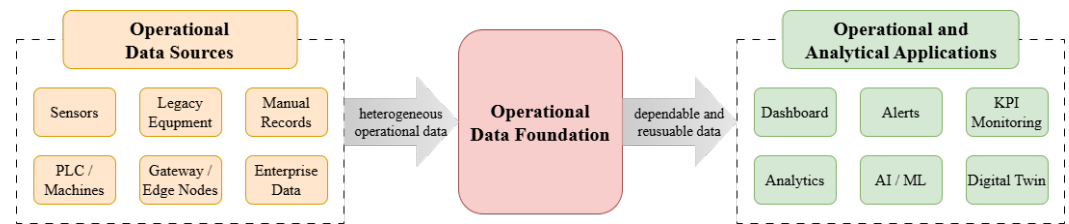


Figure 1. Position of the operational data foundation in a digitalized manufacturing system.

Within this shift, data are no longer merely retrospective records. They have become a core operational resource for visibility, traceability, quality control, maintenance, and data-driven decision making. As a result, the practical advancement of smart manufacturing increasingly depends on whether operational data can be captured, interpreted, and reused in a stable and sustained manner.

Higher-level intelligent functions do not stand on their own. Predictive analytics, digital twins, and AI-driven optimization all depend on an underlying operational layer that can continuously capture field data, preserve their meaning across system boundaries, and support downstream reuse. The key question, therefore, is not only what manufacturing systems can do intelligently, but what data conditions must first be maintained for such intelligence to become operationally viable.

The value of manufacturing data is not realized at the moment of generation alone. It emerges across an end-to-end flow spanning acquisition, transmission, transformation, storage, and downstream use. Manufacturing data problems should therefore be understood not as isolated database or interface issues, but as lifecycle-wide operational systems problems. This challenge is intensified by the intrinsic heterogeneity of industrial environments, where diverse devices, protocols, timestamps, machine states, manual records, and fragmented platforms coexist. Under such conditions, missingness, duplication, delay, semantic inconsistency, and timestamp ambiguity directly undermine traceability, operational visibility, and decision quality.

What is required, therefore, is not merely an ingestion infrastructure, but an operational layer capable of sustaining continuity, governed semantics, durable persistence, observability, and downstream reusability. In this study, we refer to this layer as the *operational data foundation*. Rather than serving as a simple conduit for data movement, it stabilizes heterogeneous field data and makes them dependable for sustained operational and analytical use. As illustrated in Figure 1, this layer occupies the space between heterogeneous operational data sources and higher-level operational and analytical applications.

Although building dependable manufacturing data systems is challenging in general, its foundational requirements become especially visible in constrained small and medium-sized enterprise (SME) factories. These environments rarely offer the conditions often assumed in digitally mature smart-manufacturing settings, such as standardized interfaces, stable internal connectivity, dedicated technical support, and integrated data infrastructure. Instead, they more often combine legacy equipment, fragmented systems, mixed communication technologies, manual records, limited technical staffing, and strong cost sensitivity. For this reason, SME factories should not be regarded merely as smaller-scale plants, but as critical design settings in which the hidden assumptions of smart manufacturing data systems are exposed most clearly.

Under such conditions, the practical bottleneck of digitalization lies not simply in deploying more sensors, collecting more data, or adding downstream applications. Rather, it lies in whether operational data can be captured with continuity, stabilized with interpretable meaning, and sustained as a usable resource despite fragmentation, instability, and limited operational slack. In SME manufacturing, the central difficulty is therefore not

merely data scarcity, but the fragility of the operational data lifecycle through which field data must become dependable for ongoing operational use.

This implies that the primary bottleneck of smart manufacturing in constrained SME factories lies not in downstream intelligence, but in establishing an operationally viable data foundation. Accordingly, this study addresses the following problem: *how should an operational data foundation be designed, and what must it guarantee, in order to make smart manufacturing practically viable in constrained SME factories?*

Existing research has addressed manufacturing data systems from several complementary directions. Smart manufacturing studies have primarily focused on higher-level applications such as real-time monitoring, predictive maintenance, digital twins, and adaptive production support. Research on industrial data infrastructures has examined pipelines, integration architectures, edge–cloud coordination, governance, and data quality. Lifecycle-oriented work has emphasized lifecycle-wide linkage, traceability, contextual continuity, and the preservation of reusable records across stages. Meanwhile, SME digitalization studies have clarified the realities of constrained deployment, including legacy equipment, fragmented systems, limited staffing, and cost sensitivity.

However, these contributions remain only partially integrated from the perspective of operational viability under constrained deployment conditions. Prior work has often addressed applications, architectural components, lifecycle properties, or SME constraints separately, rather than formalizing the operational guarantees that a manufacturing data foundation itself must provide in order to remain usable in practice. As a result, there is still limited systems-level articulation of how such guarantees can be defined, structured, and assessed for constrained SME manufacturing environments.

To address this gap, this study focuses on the foundational layer presupposed by smart manufacturing applications and formalizes it in terms of operational requirements, design principles, and assessable invariants. More specifically, it develops an *Operational Data Foundation Framework* for constrained SME factories and examines it not only conceptually, but through actual field implementation and operational evidence from a real SME manufacturing deployment.

On this basis, the study is guided by the following research questions:

- **RQ1.** What operational risks emerge when smart manufacturing data pipelines are deployed under constrained SME factory conditions?
- **RQ2.** What core requirements must an operational data foundation satisfy in such conditions, and how can they be formalized into design principles and operational invariants?
- **RQ3.** How can the proposed framework be instantiated in a real SME manufacturing setting, and how can its practical value be examined through field evidence?

To answer these questions, the study proceeds in four steps. It first establishes a general lifecycle model of smart manufacturing data pipelines as an analytical reference. It then analyzes how recurrent constraints documented in prior SME research act across that lifecycle and derives six operational risks that undermine practical viability. On that basis, it formalizes the Operational Data Foundation Framework by translating those risks into core requirements, design principles, and operational invariants. Finally, it instantiates the framework through actual field implementation in a real SME manufacturing setting and examines its practical value through requirement-based evidence and representative field observations.

The primary contributions of this research are as follows:

1. Reframing the practical bottleneck of smart manufacturing in constrained SME factories as an operational data foundation problem.

2. Deriving the core operational risks that emerge when a general manufacturing data lifecycle is subjected to constrained SME factory conditions.
3. Formalizing the Operational Data Foundation Framework in terms of core requirements, design principles, and operational invariants.
4. Demonstrating the framework's field-grounded operational relevance through real-world implementation in an SME manufacturing environment.

The remainder of this paper is organized as follows. Section 2 reviews the related literature. Section 3 analyzes the general data pipeline lifecycle, SME-specific constraints, and the resulting operational risks. Section 4 presents the proposed Operational Data Foundation Framework. Section 5 describes the field implementation in an SME manufacturing setting. Section 6 evaluates the framework using requirement-based evidence and field observations. Finally, Section 7 concludes the paper.

2. Related Work

2.1. Smart Manufacturing Applications

Smart manufacturing research has developed mainly around higher-level applications and intelligent capabilities, including industrial IoT, cyber-physical systems, digital twins, real-time monitoring, predictive maintenance, and AI/ML-enabled decision making [1–4]. Across these streams, the dominant goal has been to make manufacturing systems more connected, adaptive, and data-driven through improved visibility, prediction, and control [1,2,5].

Despite their diversity, these applications share a common prerequisite: continuous or near-real-time operational data from the shop floor [6–8]. Studies on digital twins and real-time manufacturing intelligence repeatedly show that higher-level functions depend on accurate, timely, and interpretable data aligned with physical system states [6,9,10]. Yet this literature is generally more explicit about what smart manufacturing applications can optimize, predict, or automate than about what kind of data basis must first be sustained for those functions to remain operationally viable in practice [1,6,11].

2.2. Industrial Data Infrastructures

A substantial body of research has examined the industrial data basis through manufacturing data pipelines, integration architectures, industrial data management systems, and edge-cloud infrastructures [12–14]. This literature makes clear that manufacturing data handling is not a single storage problem, but a multi-layer systems problem spanning acquisition, communication, processing, storage, and downstream delivery across heterogeneous sources [12,15].

Within this stream, two themes are especially prominent. First, many studies address heterogeneous integration and interoperability across legacy equipment, devices, protocols, platforms, and enterprise systems [16–18]. Second, related work has emphasized semantics, context, and governance, arguing that industrial data must remain interpretable, trustworthy, and reusable rather than merely transferable [19–21]. These contributions have significantly advanced the understanding of how industrial data infrastructures can be connected, organized, and managed. However, they remain more focused on components, architectures, and enabling techniques than on explicitly formalizing what such infrastructures must guarantee as an operational foundation under deployment constraints [12,13,20].

2.3. Lifecycle Connectivity

Another important line of research treats manufacturing data not as stage-local records but as lifecycle-wide resources. From this perspective, the value of manufacturing data

depends on whether information can remain linked, contextualized, and reusable across design, production, inspection, operation, maintenance, and end-of-life stages [22–25]. This view shifts attention from isolated data handling toward continuity of meaning and usability across lifecycle transitions.

The concept of the digital thread has become the clearest expression of this perspective. Recent reviews and implementation studies define the digital thread as a means of linking information across heterogeneous lifecycle artifacts and stages so that traceability, cross-stage visibility, and lifecycle-wide reuse become possible beyond fragmented system silos [26–29]. Related studies on contextualization, standards-based linkage, and knowledge-graph integration likewise show that lifecycle usefulness depends not merely on storing records, but on preserving associativity, context, and interpretability for later reuse [30–32]. At the same time, this literature has concentrated mainly on linkage and traceability, leaving relatively less explicit discussion of how such continuity and reuse can be sustained operationally under constrained deployment conditions.

2.4. SME Deployment

The SME literature makes those deployment constraints explicit. Studies on smart manufacturing adoption in SMEs consistently show that these environments differ substantially from idealized smart factory assumptions: legacy equipment, fragmented systems, uneven interfaces, limited digital infrastructure, restricted investment capacity, and uneven technical expertise are common [33–36]. In this sense, SMEs should not be viewed simply as smaller large factories, but as constrained deployment settings in which the assumptions of digitally mature architectures are most directly challenged.

This constraint is both technical and operational. Brownfield dependence often forces SMEs to rely on retrofitting, add-on sensing, gateway-based integration, and low-cost connectivity rather than wholesale replacement [37–40]. At the same time, recurrent barriers include limited staffing, weak digital skills, uncertain return on investment, high implementation cost, and maintainability burden [41–44]. Accordingly, SME-oriented studies increasingly advocate phased, modular, and capability-building adoption paths [36,45,46]. Taken together, these studies show that the unresolved problem is not how to deploy smart manufacturing application, infrastructure component, or lifecycle linkage mechanism, but how to sustain a data foundation that remains operationally viable under constrained real-world conditions.

3. Problem Analysis

This section analyzes the data basis of smart manufacturing as an operational pipeline and examines how constrained SME factory conditions impose structural fragility across that pipeline. Rather than merely describing deployment difficulty, it identifies how lifecycle-wide constraints converge into a small set of recurring operational risks. In this way, the section defines the problem structure of operational data foundations under constrained deployment conditions.

3.1. General Data Pipeline Lifecycle

A common analytical reference is required before constrained SME conditions and their resulting risks can be examined in a structured way. In smart manufacturing, data do not become operationally useful at the moment of generation alone. Their practical value emerges only through a broader path in which field observations are accepted into a pipeline, transferred across infrastructure layers, transformed into usable forms, preserved, and connected to downstream operational use. For this reason, this study adopts a *general*

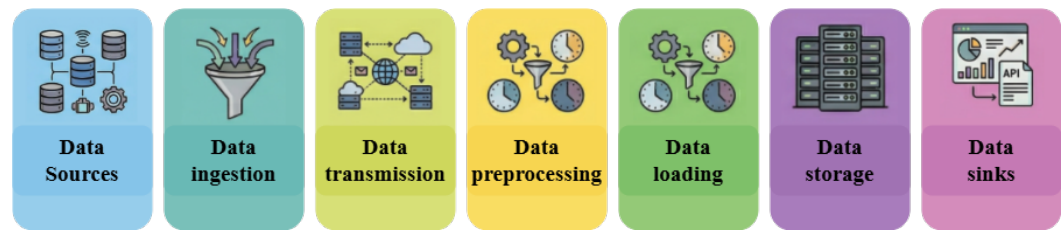


Figure 2. General data pipeline lifecycle in smart manufacturing.

data pipeline lifecycle as an analytical frame for problem interpretation rather than as a normative system architecture.

The lifecycle used in this study is shown in Figure 2. It is adapted from generic data pipeline literature that treats data systems as multi-stage processing structures with recurring issues around ingestion, integration, cleaning, transformation, loading, and pipeline-wide quality control [47]. In the smart manufacturing context, this general view is organized as follows:

- **Data sources:** the points at which operational data originate, including sensors, equipment states, manual records, and external systems.
- **Data ingestion:** the intake of source data into the pipeline.
- **Data transmission:** the movement of acquired data across gateway, edge, server, or cloud boundaries toward the next processing point.
- **Data preprocessing:** the cleaning, decoding, normalization, mapping, and transformation required to make raw inputs usable downstream.
- **Data loading:** the controlled insertion of processed records into the target operational layer or storage boundary.
- **Data storage:** the durable preservation of records for later access, recovery, reinterpretation, and reuse.
- **Data sinks:** the downstream endpoints that consume stored data, including dashboards, alerts, reporting, analytics, and decision support.

This lifecycle is analytically useful because it makes visible the full path through which manufacturing data become operationally usable. At the same time, it is not a design answer on its own. It describes how data move, but not what must be preserved so that the lifecycle remains operationally viable under constrained conditions. It is therefore used in this study as an analytical reference for the constraint and risk analysis that follows.

3.2. SME Constraints

Constrained SME factories should not be treated simply as smaller versions of digitally mature plants. Prior research shows that their starting conditions are structurally different: legacy equipment, fragmented systems, uneven interfaces, restricted digital infrastructure, limited internal expertise, and strong investment sensitivity are common, whereas standardized connectivity and unified data access are often absent [33–36,48]. SME constraints are therefore better understood not as isolated implementation difficulties, but as recurring design conditions acting across the general data pipeline lifecycle. Figure 3 highlights this point by showing how the major constraint types propagate across source capture, transfer, transformation, storage, and downstream use rather than remaining confined to a single stage.

Heterogeneity begins at the source side. In SME factories, brownfield machinery, retrofit-based extension, add-on sensing, and gateway-mediated access are common, so inconsistency is already present at the level of data sources and ingestion rather than appearing only later in integration [37–40]. The challenge is not merely collecting values, but incorporating diverse formats, units, reporting intervals, and source identities into a co-

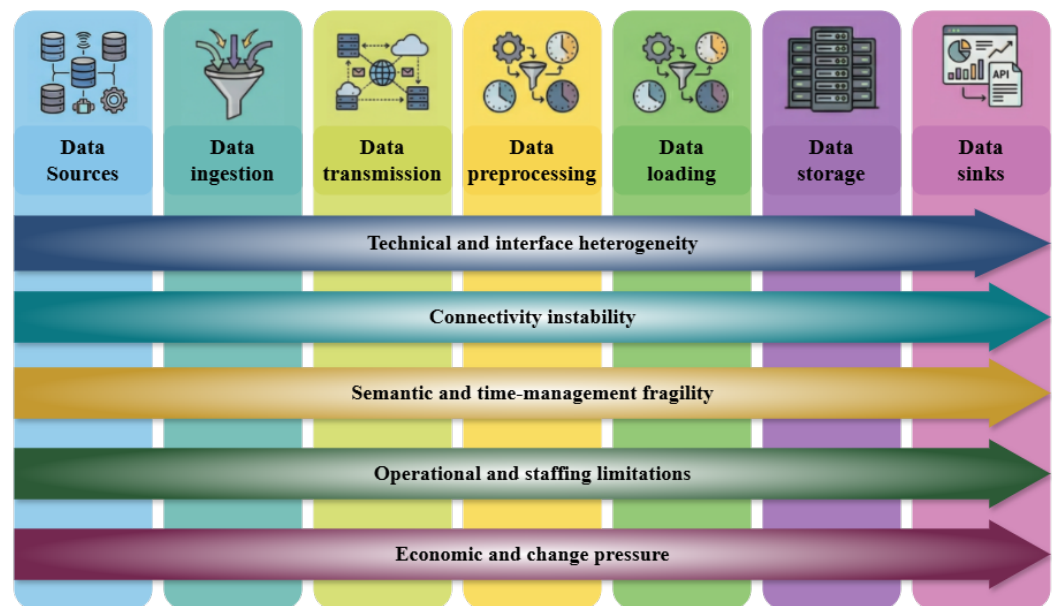


Figure 3. SME constraints across the general data pipeline lifecycle.

herent operational structure. Source-side variability is thus not a peripheral inconvenience, but the starting condition from which later coordination and interpretation problems arise.

Transmission is often fragile in constrained SME deployments. In constrained SME deployments, field data may depend on low-cost wireless communication, battery-powered sensing, and gateway- or edge-dependent relay paths rather than on stable plant-wide infrastructure [49–51]. The ingestion–transmission path therefore becomes a distinct surface of instability. What matters operationally is not simply whether a value was observed once, but whether the record can remain sufficiently continuous and recoverable despite interruption, delay, and relay dependence.

Operational meaning must remain stable across the pipeline. Manufacturing data are operationally usable only when source identity, metric semantics, unit interpretation, and time semantics stay controlled across preprocessing, loading, and storage. Yet constrained environments frequently rely on heterogeneous preprocessing logic, ad hoc mappings, incomplete context, and weak contract discipline, making semantic drift and traceability loss structurally likely [17,19–21,52]. Once such instability becomes embedded in stored records, downstream analytics and dashboards may continue to function while resting on an increasingly fragile semantic basis.

These technical difficulties are intensified by operational reality. SME studies repeatedly report limited staffing, uneven digital skills, manual troubleshooting, high maintenance burden, and weak strategic capacity [41,43,44,53]. At the same time, SME digitalization rarely proceeds as a one-shot transformation; it unfolds through phased rollout, retrofit-based extension, KPI revision, and incremental onboarding of new sources and processes [45,46,54,55]. As a result, the practical challenge is not only whether a system can be built, but whether it can remain diagnosable, manageable, and adaptable as requirements change.

Taken together, these observations show that the core problem in constrained SME factories is not simply that deployment is difficult. Rather, the assumptions of the general data pipeline lifecycle are systematically destabilized across the lifecycle itself: records may lose continuity, meanings may drift, failures may become hard to localize, operational burden may grow, historical data may lose replay value, and later extension may require disproportionate rework. The next subsection translates these recurring forms of fragility into the language of operational risk.

3.3. Operational Risks

The SME constraints discussed above do not remain as isolated deployment difficulties. When mapped onto the general data pipeline lifecycle, they appear as recurring operational risks that determine whether manufacturing data can remain usable in practice. The issue is therefore not only how data move through the pipeline, but what must be preserved so that the resulting record remains dependable under constrained real-world conditions.

- RSK1. Continuity risk.** Operational records may become interrupted, fragmented, or partially lost as data move from source capture to storage. In constrained manufacturing settings, the essential requirement is not one-time collection success, but the preservation of a gap-minimized and recoverable record despite source instability, transmission disruption, or delayed recovery.
- RSK2. Governance risk.** Data may remain available while losing stable meaning. This occurs when source identity, metric semantics, timestamp interpretation, or transformation logic are not kept consistent across collection, preprocessing, loading, and storage. Under such conditions, downstream analysis may continue to operate, but on data whose meaning is no longer reliably governed.
- RSK3. Diagnosability risk.** When failure occurs, it may be difficult to determine where in the lifecycle the problem emerged and what kind of failure it is. Weak monitoring, poor stage separation, and limited observability turn degradation into a black-box symptom, making it difficult to distinguish transmission loss, semantic corruption, storage inconsistency, or downstream mismatch in a structurally interpretable way.
- RSK4. Operability risk.** A system may function technically while becoming too costly or complex to sustain in everyday practice. Tight coupling, heavy maintenance burden, unclear ownership, and non-modular structures raise the effort required to manage, modify, and repair the system, making long-term operation difficult under constrained SME staffing and budget conditions.
- RSK5. Reprocessability risk.** Historical data may cease to be usable when rules, KPIs, schemas, or interpretation logic change. If raw preservation is weak, preprocessing is effectively irreversible, or lineage is incomplete, past records cannot be reliably replayed, recomputed, or reinterpreted. In that case, the data foundation loses one of its most important operational values: the ability to support correction and renewed interpretation over time.
- RSK6. Evolvability risk.** New sensors, processes, lines, factories, or downstream requirements may become difficult to incorporate without disproportionate redesign cost or structural disruption. Since SME digitalization usually proceeds through incremental onboarding rather than one-shot integration, a data foundation that cannot absorb change without breaking governance and operational control cannot serve as a durable basis for growth.

Taken together, these risks show that a smart manufacturing data pipeline in constrained SME factories cannot be treated as a simple ingestion flow. The problem is not a collection of isolated device, network, or software issues, but a structural fragility of the lifecycle through which operational data become usable. What is required, therefore, is a foundation that can explicitly preserve continuity, governance, diagnosability, operability, reprocessability, and evolvability under constrained deployment conditions. The next section presents the *Operational Data Foundation Framework* as a structured response to these risks.

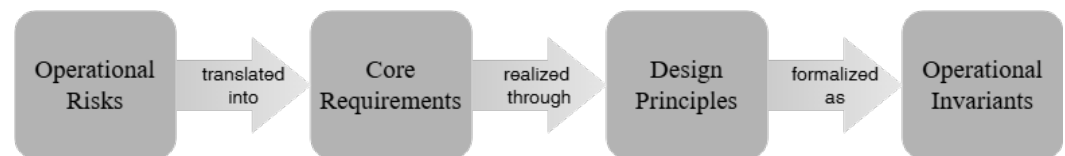


Figure 4. Operational Data Foundation Framework.

4. Operational Data Foundation Framework

4.1. Framework Overview

Section 3 showed that, in constrained SME factory environments, smart manufacturing data pipelines are exposed to recurring risks of continuity loss, semantic instability, weak diagnosability, operational burden, limited reprocessability, and poor evolvability. These risks indicate that the data layer cannot be treated merely as a transmission path or collection mechanism. What is needed instead is an explicit framework that defines what an operational data foundation must guarantee in order to remain viable under constrained deployment conditions.

To address this need, this study proposes the *Operational Data Foundation Framework*. As shown in Figure 4, the framework is organized as a risk-to-design logic: *operational risks* are translated into *core requirements*; these requirements are realized through *design principles*; and their realized state is assessed through *operational invariants*. In this way, the framework converts the problem structure identified in Section 3 into an explicit and assessable basis for the design and evaluation of an operational data foundation.

4.2. Core Requirements

The framework begins by defining the *core requirements* of an operational data foundation. These requirements are not desirable properties in a general sense, but the minimum lifecycle-wide guarantees needed for operational viability under constrained SME factory conditions. They provide the basis for the design principles and operational invariants developed in the following subsections.

Six core requirements are defined.

- R1. Continuity.** Data continuity should be preserved as far as practicable despite field disruption and connectivity instability. The essential requirement is not momentary acquisition success, but the minimization of gaps and the preservation of recoverable continuity so that the operational record remains usable even when intermediate segments are unstable.
- R2. Governance.** Source identity, metric semantics, and time semantics should remain stable across collection, normalization, and storage. Data are not operationally usable merely because they exist; they must retain controlled meaning and interpretable context throughout the pipeline.
- R3. Diagnosability.** Failures should be observable as segment-level discrepancies rather than only as end-to-end symptoms. The requirement is not simply that problems become visible after damage appears downstream, but that the lifecycle location and mode of failure can be identified in a structurally interpretable way.
- R4. Operability.** The system should remain maintainable and manageable under constrained staffing and budget conditions. An operational data foundation must therefore be sustainable in day-to-day practice without imposing structurally excessive maintenance burden or operational complexity.
- R5. Reprocessability.** Historical data should remain reprocessable for rule changes, KPI revisions, correction, and recovery. Manufacturing data should not be treated as one-

time consumables, but as records that can be read again, recalculated, and reinterpreted when operational needs evolve.

- R6. Evolvability.** The foundation should accommodate new sources, processes, sites, and applications without breaking the existing structure. Since SME digitalization typically proceeds through incremental onboarding rather than one-time integration, the data foundation must remain structurally open to change and extension.

4.3. Design Principles

The core requirements defined above specify what the operational data foundation must guarantee, but not how those guarantees should be realized in design. This subsection therefore presents the key *design principles* that translate those requirements into a workable design logic for constrained SME factory conditions. Rather than prescribing specific technologies or implementation choices, these principles define the structural directions needed to sustain the required operational guarantees.

- P1. Continuity responsibility should be placed close to the source.** If continuity depends primarily on downstream transmission or cloud reachability, it becomes fragile under field disruption and connectivity instability. For this reason, the first responsibility for preserving continuity should be located as close to the source as possible through source-near capture, upstream buffering or persistence, and recovery-oriented ingestion logic.
- P2. Raw events should be preserved before abstraction.** If only transformed outputs are retained, later replay, reinterpretation, and recovery become structurally limited. The preservation of raw events before irreversible abstraction is therefore a fundamental design principle, since rule changes, KPI revision, and correction all depend on access to preserved pre-abstraction records.
- P3. Semantics should be contract-defined.** If source identity, metric naming, timestamp semantics, unit definition, and transformation logic are left to ad hoc implementation, semantic inconsistency accumulates across the pipeline. The design must therefore define semantics explicitly through controlled contracts, schema discipline, version-aware rules, and clear identity and time conventions.
- P4. Observability should follow pipeline segments.** Diagnosability cannot be secured through black-box end-to-end monitoring alone, because operationally meaningful failures must be localizable to specific lifecycle segments. Observability should therefore be structured along the pipeline itself through segment-wise checkpoints, stage-level indicators, and explicit discrepancy visibility.
- P5. Responsibilities should be separated by function.** If acquisition, normalization, storage, and downstream use are tightly coupled, both maintenance burden and change cost rise rapidly. Functional responsibilities should therefore be separated so that components remain structurally independent, interfaces remain replaceable, and modifications in one part do not unnecessarily destabilize the whole.
- P6. Operations should remain lightweight, modular, and open.** In constrained SME environments, heavy operational structures, costly maintenance models, and vendor dependence undermine long-term viability. The design should therefore remain lightweight in deployment, modular in composition, manageable in maintenance scope, and open to replacement and extension. Where appropriate, open and widely supported components, including open-source tools, can help reduce lock-in and maintenance burden.
- P7. Change should be treated as a first-class assumption.** Changes in KPIs, schemas, rules, source onboarding, and deployment scope are not exceptional events but normal operating conditions in SME digitalization. The foundation should therefore be

designed from the outset to tolerate and organize change through version-aware structures, replay support, incremental onboarding discipline, and change-tolerant interfaces.

4.4. Operational Invariants

The core requirements and design principles defined above specify what the operational data foundation must guarantee and how those guarantees should be realized. This subsection completes the framework by defining the *operational invariants*: the persistent conditions that should hold if the foundation is functioning as intended.

- I1. Durable Capture Invariant.** An accepted event should obtain durable persistence before downstream failure can erase it. Continuity should therefore be judged from the point of first durable capture rather than from downstream availability alone.
- I2. Traceability Invariant.** Every canonical or stored record should remain traceable to its source identity and operational context. Downstream data should not become detached from the source, metadata, and governing semantics that give it meaning.
- I3. Deterministic Normalization Invariant.** The same raw event under the same contract and version conditions should always produce the same canonical output. Normalization should function as a repeatable and controlled semantic transformation rather than as ad hoc interpretation.
- I4. Segment-Localizable Failure Invariant.** A failure should become visible as at least one discrepancy within a recognizable lifecycle segment. It should be possible to attribute the problem to a specific segment such as source, transmission, preprocessing, loading, storage, or sink rather than observing it only as a downstream symptom.
- I5. Replayability Invariant.** Preserved raw data should remain reprocessible for rule changes, KPI revision, correction, and recovery. Historical data should therefore remain usable for later reinterpretation and recomputation rather than serving only as passive archive material.
- I6. Modular Replacement Invariant.** Replacing a specific component should not break the semantic chain of the overall foundation. Changes in storage, parsing, transport, or downstream components should not destroy source identity, metric meaning, or replay context across the system.
- I7. Onboarding Consistency Invariant.** A new source should be incorporable into the existing structure without breaking the prevailing governance discipline. Incremental onboarding should proceed through the same contract, schema, naming, and validation discipline rather than through an accumulation of ad hoc exceptions.

These invariants define the persistent conditions by which the framework can be assessed in practice. If requirements declare the guarantees to be achieved and principles define the design logic needed to achieve them, invariants provide the basis for evaluating whether those guarantees remain structurally preserved.

4.5. Framework Synthesis

This subsection synthesizes the framework by reconnecting it to the problem space analyzed in Section 3. Section 3 showed how constrained SME factory conditions destabilize the general data pipeline lifecycle and generate recurring risks of continuity, governance, diagnosability, operability, reprocessability, and evolvability. Sections 4.2–4.4 translated those risks into core requirements, design principles, and operational invariants. The purpose here is to clarify how these layers are systematically linked to the original problem structure.

Table 1 provides this first synthesis. It shows that constrained SME conditions are not merely a list of deployment difficulties, but recurring operational design problems that

can be translated in a structured way. Each major constraint category gives rise to one or more operational risks; each risk is then expressed as a core requirement, translated into one or more design principles, and made assessable through corresponding invariants. The table thus shows how the framework is grounded in recurring structural problems in constrained SME manufacturing.

Table 1. From SME constraints to operational design logic.

SME Constraint	Operational Risk	Requirement	Design Principle	Invariant
Heterogeneous devices and fragmented interfaces	RSK2 (Governance risk)	R2 (Governance)	P3 (Semantics must be contract-defined)	I2 (Traceability), I3 (Deterministic normalization)
Add-on sources and partial visibility	RSK2, RSK6 (Evolvability risk)	R2, R6 (Evolvability)	P3, P7 (Change as a first-class assumption)	I2, I7 (Onboarding consistency)
Unstable connectivity and constrained sensing	RSK1 (Continuity risk)	R1 (Continuity)	P1 (Continuity responsibility must be placed close to the source)	I1 (Durable capture)
Weak cross-segment observability	RSK3 (Diagnosability risk)	R3 (Diagnosability)	P4 (Observability must follow pipeline segments)	I4 (Segment-localizable failure)
Limited staff and high maintenance burden	RSK4 (Operability risk)	R4 (Operability)	P6 (Operations must remain lightweight, modular, and open)	I6 (Modular replacement)
Weak raw-data preservation	RSK5 (Reprocessability risk)	R5 (Reprocessability)	P2 (Raw events must be preserved before abstraction)	I5 (Replayability)
Tight functional coupling across the pipeline	RSK3, RSK4, RSK6	R3, R4, R6	P5 (Responsibilities must be separated by function)	I4, I6
Frequent KPI, rule, schema, and source changes	RSK5, RSK6	R5, R6	P7	I5, I7
Budget sensitivity and lock-in concerns	RSK4, RSK6	R4, R6	P6	I6, I7

One particularly important translation highlighted in Table 1 is the movement from semantic and interface heterogeneity to governance logic. In constrained SME factories, heterogeneous devices, mixed protocols, fragmented interfaces, add-on sensing, and manual records do not merely create diverse formats; they destabilize source identity, metric meaning, unit interpretation, and timestamp semantics. The issue is therefore not simply one of integration, but one of governance. For this reason, the framework connects this problem to *R2 Governance*, *P3 contract-defined semantics*, and the assessable conditions of *I2 Traceability* and *I3 Deterministic Normalization*. This translation makes clear that manufacturing data become operationally reliable only when meaning and origin remain stably governed.

A second important translation concerns the movement from change pressure to reprocessability and evolvability logic. SME digitalization rarely proceeds as a one-time integration event; instead, KPI definitions evolve, schemas change, new sensors and lines are added, and operational rules are revised over time. When such change is not treated as a normal operating condition, transformed-only persistence and weak lineage make past data difficult to reinterpret and new sources difficult to onboard without exception-heavy

modification. The framework therefore connects this problem to *R5 Reprocessability* and *R6 Evolvability*, supported by *P7 change as a first-class assumption* and assessed through *I5 Replayability* and *I7 Onboarding Consistency*. This shows that the framework is designed not only for present-time correctness, but also for preserving operational coherence under future change.

Table 2 provides a second synthesis by showing how the framework adds operational meaning to the general data pipeline lifecycle. Under constrained SME conditions, each lifecycle stage must contribute not only to data movement and processing, but also to the guarantees defined by the operational data foundation. The table makes this added meaning explicit by connecting each stage to the relevant risks, requirements, design principles, and invariants. In this way, the general lifecycle is retained as an analytical reference, while its role is sharpened for constrained manufacturing conditions.

Table 2. Mapping the general data pipeline lifecycle to the operational foundation logic.

General Lifecycle Stage	Added Operational Meaning	Related Risks	Requirements	Principles	Invariants
Data sources	Stable source identity	RSK2, RSK6	R2, R6	P3, P7	I2, I7
Data ingestion	Source-near continuity and intake control	RSK1, RSK2	R1, R2	P1, P3	I1, I2
Data transmission	Recoverable transfer and visible discrepancies	RSK1, RSK3	R1, R3	P1, P4	I1, I4
Data preprocessing	Contract-defined and deterministic transformation	RSK2, RSK6	R2, R6	P3, P7	I2, I3, I7
Data loading	Controlled persistence with replay context	RSK5, RSK6	R5, R6	P2, P7	I5, I7
Data storage	Replay-ready and replaceable persistence	RSK4, RSK5	R4, R5	P2, P6	I5, I6
Data sinks	Governed downstream reuse	RSK2, RSK5	R2, R5	P3, P7	I2, I5
Monitoring and management across the lifecycle	Lifecycle-wide observability and maintainability	RSK3, RSK4	R3, R4	P4, P6	I4, I6

One especially revealing example is the reinterpretation of the transmission stage. In generic pipeline discussions, transmission is often treated as a routine internal link between ingestion and later processing. In constrained SME factory conditions, however, the path from field devices to gateways, edge systems, servers, and cloud systems is often exposed to intermittent connectivity, limited bandwidth, battery constraints, delayed recovery, and relay dependence. Transmission therefore becomes a distinct operational segment in which continuity and diagnosability are directly tested. For this reason, Table 2 assigns the transmission stage the added meanings of recoverable transfer and segment-aware

discrepancy visibility, linking it to *R1 Continuity*, *R3 Diagnosability*, *P1 source-near continuity*, *P4 segment-based observability*, *I1 Durable Capture*, and *I4 Segment-Localizable Failure*.

Taken together, Table 1 translates constrained SME conditions into operational design logic, while Table 2 shows how that logic refines the interpretation of the general data pipeline lifecycle under real manufacturing constraints. The result is a single analytical structure that connects problem space, framework logic, and lifecycle interpretation. This synthesis also provides the basis for reading the field implementation in Section 5 and the evidence assessment in Section 6.

5. Field Implementation

This section describes how the proposed framework was instantiated in a real SME manufacturing setting. The implementation is presented not as a generic IoT deployment, but as a field realization of an operational data foundation under constrained conditions. The focus is therefore placed on how heterogeneous sources were incorporated, how continuity was secured close to the source, how semantics were stabilized through the processing and storage path, and how the resulting data layer was connected to operational visibility and intervention.

5.1. Deployment Setting

The empirical setting of this study was the Idang Food yakgwa plant, one of several OEM factories managed by ONGOING, an F&B company operating across multiple production sites. The deployment was therefore designed not as an isolated factory pilot, but as the first field implementation of a broader architecture in which operational data from multiple factories could be collected locally and managed through a shared cloud-side structure. In this sense, Idang Food was selected not only as a practical implementation site, but also as a representative case through which this study examines how an operational data foundation can be established under constrained SME manufacturing conditions.

The plant's production flow included raw-material receiving, weighing, mixing, forming, frying, coating, cooling, inner packaging, detection, outer packaging, and storage. Yet the main challenge of the site did not lie simply in the number of stages. Operationally relevant states were distributed across different rooms, devices, and tasks, while production and quality management depended heavily on manual records and worker judgement. Process conditions were not accumulated as connected records, weighing-related checks remained strongly operator-dependent, and environmental histories across process and storage zones were difficult to preserve and compare over time. As a result, when abnormal conditions or quality issues emerged, the plant did not lack observations in an absolute sense; rather, it lacked continuous and reusable operational records that could support traceability and later interpretation.

This made the site suitable for the present study. Idang Food reflected several conditions that commonly characterize constrained SME factories: heterogeneous and partly non-networked sources, fragmented process visibility, limited internal connectivity, and restricted operational slack for day-to-day technical management. Under such conditions, an operational data foundation could not be assumed to arise simply from downstream software or cloud connectivity. It first had to be built from the field upward by making process states recordable, collectable, and sustainable as data under actual production constraints.

The field deployment should be understood as an actual on-site installation and system realization rather than as a conceptual design exercise. The study did not begin from an already prepared digital source environment; it proceeded by physically establishing one inside the factory through sensor placement, gateway configuration, and edge-side

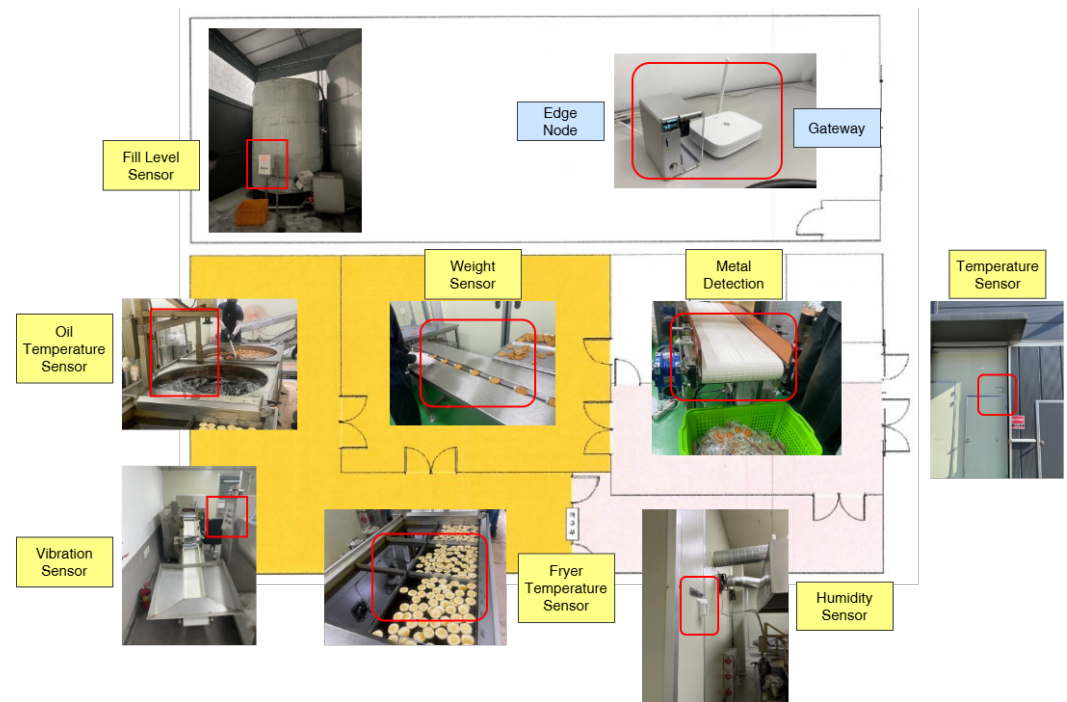


Figure 5. Factory layout with representative sensing and edge deployment.

deployment. Figure 5 presents this implementation context by showing the production environment together with representative sensing and edge elements installed in the field. This point is important because the case demonstrates not only that an operational data foundation was conceptually proposed, but that its source-side basis was concretely built and operated in a constrained SME factory. Through that installation, the plant acquired the initial operational foothold required for continuity, traceability, and later cloud-side integration.

Accordingly, the role of Idang Food in this study was twofold. At the factory level, it provided a realistic constrained environment in which fragmented sources, manual practices, and brownfield conditions had to be handled in practice. At the system level, it served as the first implementation site of an architecture intended to support factory-to-cloud data flow and centralized multi-factory management. For this reason, the next subsection begins with field data acquisition and the edge architecture, because the first requirement of the operational data foundation was to explain how field data were physically captured and operationally secured at the factory before they were forwarded into the broader cloud-side system.

5.2. Field Data Acquisition and Edge Architecture

The factory-side acquisition layer was implemented as an edge-centered intake stack that received field data inside the plant, preserved them locally, and then forwarded them to the cloud event pipeline. Because the plant did not provide a uniform network-native source environment and stable plant-wide connectivity could not be assumed across distributed process areas, the architecture was organized around a local edge boundary rather than direct source-to-cloud transmission.

LoRaWAN, a low-power wide-area communication approach well suited to distributed sensing, was selected as the field communication network in this deployment. The choice reflected the actual plant environment: the production areas did not provide wired LAN coverage, and stable WiFi-based collection could not be assumed across the factory. Under these conditions, the field network could not be designed on the assump-

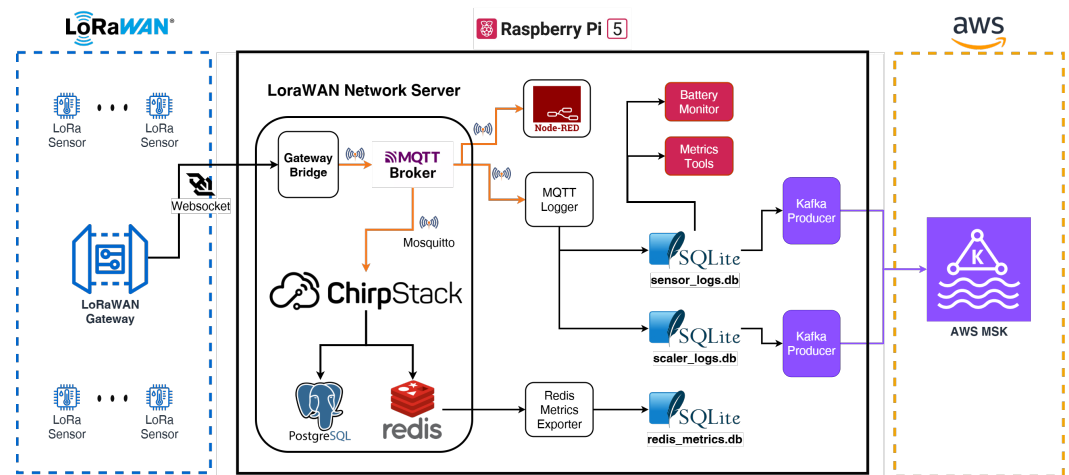


Figure 6. Edge architecture for field data acquisition.

tion of always-on internal Internet connectivity. LoRaWAN therefore provided a practical means of collecting distributed field events without extensive wiring while keeping the gateway–edge path locally controlled inside the factory.

As shown in Figure 6, the implemented architecture was organized into three zones: the in-factory field layer on the left, the internal edge stack at the center, and the cloud broker boundary on the right. Field events were transmitted through LoRa to the gateway, handled by ChirpStack Gateway Bridge and ChirpStack Server, relayed through an MQTT broker, and then accepted by edge-side services. On the edge node, MQTT logger and source-specific collectors wrote accepted records into local SQLite stores before cloud delivery was assumed. This made the acquisition path explicit—field transmission, gateway reception, LoRaWAN handling, MQTT relay, edge logging, and upper-layer forwarding were treated as separate steps—and established the edge node as the first stable intake and persistence boundary of the system.

The edge stack also accommodated heterogeneous source conditions rather than only native wireless sensor uplinks. A representative example was the production electronic scale, which did not provide native network communication and exposed only a printer port as its available output interface. To incorporate the device without replacement, that port was connected to the Raspberry Pi through an RS232 serial link. A dedicated daemon collector parsed the incoming weighing values, stored them locally, and then transmitted them through a LoRa device attached to the Raspberry Pi so that they could enter the same acquisition path as other field events. In practical terms, this flow can be summarized as *Scale* → *RS232* → *RPi* → *LoRa*. This showed that the edge architecture could absorb non-networked legacy equipment as well as standard wireless sensor uplinks within the same factory-side event flow.

Once accepted and retained at the edge, the records were forwarded by producer services to the cloud broker layer. In the broader architecture, this upstream path was organized around Kafka-based streaming, including AWS MSK on the cloud side, so that raw events from each factory could be published into a shared event pipeline for subsequent parsing, normalization, and persistence. The next subsection explains that cloud-side processing path.

5.3. Cloud Processing and Event Flow

On top of the field acquisition layer, the system implemented a cloud-side processing path that received raw events from factory edges, routed them through a brokered stream architecture, and transformed them into persistent and service-ready records. Multiple

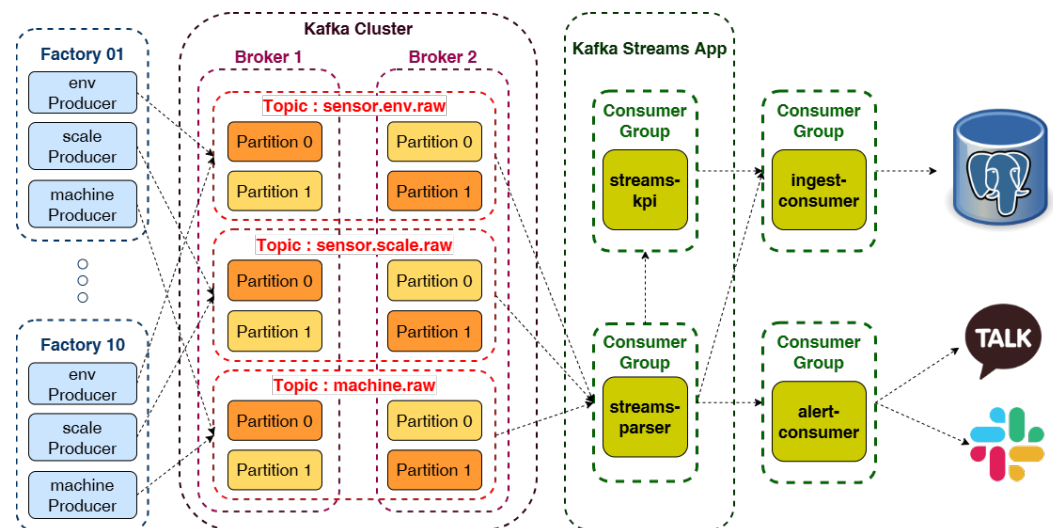


Figure 7. Cloud-side processing architecture.

factory-originated event streams were forwarded to the AWS cloud and published into Kafka topics in the MSK cluster, where subsequent stages handled normalization, derivation, persistence, and operational consumption. Figure 7 presents this deployment-level structure by showing how data from multiple factories converged into the shared AWS-based cloud environment and entered a common brokered processing path. In this way, the cloud layer served not as a single application endpoint, but as a staged processing structure connecting multi-factory intake to downstream use.

Within this cloud path, raw factory events first entered broker topics such as `sensor.env.raw`, `sensor.scale.raw`, and `machine.raw`. These raw topics were consumed by the parser stream, which decoded heterogeneous payloads, mapped sources, unified metrics, normalized timestamps, applied deduplication, and produced canonical events for downstream use. On top of this normalized stream, KPI processing and alert-side consumption proceeded as separate downstream paths: KPI streams generated derived indicators such as production quantity and yield, while alert consumers used the same governed event basis to detect conditions requiring notification or intervention. Finally, ingest consumers persisted the normalized and derived outputs into the database layer for later dashboard, alert, and analytical use. Figure ?? clarifies this processing sequence by showing how the input and output of each stage changed as raw messages were progressively transformed into reusable operational records.

A central design objective of this layer was functional decoupling. Factory-side producers were allowed to publish raw events to the broker without depending directly on parser execution, KPI computation, alert handling, or database ingestion. This separation reduced failure propagation across stages and made the system easier to modify as rules, sources, and downstream requirements evolved. In practical terms, the broker layer provided the shared event boundary through which intake, interpretation, derivation, persistence, and service consumption could remain structurally separated.

The parser stream concentrated the semantic normalization responsibility of the cloud path. Rather than allowing later components to interpret device-specific payload structures independently, the system converted heterogeneous source messages into a governed common form at this stage. In this implementation, `source_id` functioned as the canonical identifier of each operational source by organizing identity around controlled attributes such as tenant, line, process, device type, and metric, while timestamp semantics were stabilized by distinguishing `event_time`, `edge_ingest_time`, and `db_ingest_time` as separate time axes with different operational roles. This prevented ambiguity in ordering

and interpretation and kept downstream processing aligned with a consistent semantic discipline.

After canonicalization, KPI computation was intentionally performed in the stream layer rather than deferred to dashboard query time. If production quantity, yield, and related indicators had been computed repeatedly from raw or lightly processed records at query time, each view request would have required additional aggregation and business-rule evaluation over a larger event set. By computing these indicators continuously from the normalized stream and persisting the resulting outputs as ready-to-use records, the system shifted repeated query-time work into incremental event-time processing. This reduced dashboard-side computational burden, simplified query logic, and supported more stable low-latency operational views. At the same time, alert consumers could react to the same governed event basis without overloading either the parser or the persistence layer.

Persistent storage was handled through ingest consumers. These consumers received already normalized or derived events and wrote them into the database without reintroducing business logic at the persistence boundary. As a result, semantic interpretation remained in the parser layer, derived computation remained in the KPI stream, alert-side reaction remained in dedicated consumers, and the storage path remained focused on durable append-oriented persistence. This responsibility split made the cloud-side stack easier to reason about and more robust to later changes in processing rules or downstream interfaces.

Taken together, the cloud-side processing path completed the second half of the operational data foundation. If the field edge established source-near acceptance and local durability, the cloud layer provided brokered multi-factory intake, governed normalization, separated derivation and alert handling, and service-ready persistence. The result was a processing architecture that did not expose raw heterogeneous payloads directly to downstream services, but converted them into controlled operational records that could support dashboards, alerts, traceability, and future analytical extension.

Table 3. Cloud-side components and responsibilities.

Component	Input	Output	Primary responsibility
Producer	Edge-side records/messages	Raw topics	Forward field data to the cloud event flow
Parser stream	Raw topics	Canonical events	Decode, normalize, map sources, and standardize events
KPI stream	Canonical events	KPI events	Derive production- and process-level indicators
Ingest consumer	Canonical events, KPI events	Database records	Persist processed events without adding business logic
Database	Ingested records	Queryable tables/views	Provide persistent storage for dashboards, alerts, and analysis
Dashboard/alert layer	Stored and derived data	Operational interfaces	Present status, alarms, and management views

5.4. Operational Use

The implementation extended beyond data capture and cloud-side processing into an operational layer through which plant personnel could observe conditions and respond when necessary. This layer mattered because the system was not intended to remain a background pipeline. It was designed to make governed operational records directly usable in day-to-day manufacturing practice and thereby connect the data foundation to actual plant awareness and intervention.

Dashboards translated processed records into operational views suitable for both operators and managers. Instead of exposing raw signals alone, the system presented current conditions, recent trends, source health, packet reception quality, and derived indicators in forms that supported routine monitoring. This changed the practical mode of work at the plant. Before digitalization, condition checking depended heavily on manual records, fragmented local observation, and worker experience. With the implemented layer, plant personnel could instead rely on a shared and continuously updated reference for process and environmental conditions, reducing exclusive dependence on tacit judgement and making operational awareness more consistent and reviewable.

Alerting formed the intervention path of this layer. When operationally meaningful conditions were detected—for example, fryer oil temperature departing from its expected range, storage-area temperature rising sharply during extreme summer heat, prolonged silence from a source, or battery-related degradation affecting sensor reliability—the system generated asynchronous alerts so that personnel could respond in time. In this design, alerts were not an accessory to dashboards, but the mechanism through which monitored states became actionable events. The resulting loop can be understood as observe–detect–notify–act.

The implemented layer also reached the point of machine-side intervention capability. The system was built to support actual device power control in the field, showing that the operational data foundation did not remain limited to passive observation and notification. Although this study does not center on closed-loop control automation itself, this capability is important because it shows that the same governed data basis could already support downstream action as well as monitoring. Taken together, the operational layer demonstrates that the implemented system functioned not merely as a data pipeline, but as a practical interface connecting visibility, alerting, human response, and intervention on a common operational data foundation.

6. Evaluation of the Framework

This section evaluates whether the proposed framework functioned as an operational data foundation in a real manufacturing setting. Rather than emphasizing benchmark performance or algorithmic accuracy, the evaluation examines how the six framework requirements became visible in the implemented system and its operational history. Accordingly, Section 6.1 reviews requirement-wise evidence, and Section 6.2 presents a representative field case to illustrate more concretely how the framework became visible in actual operation.

6.1. Framework Compliance

This subsection examines framework compliance across the six core requirements defined in Section 4. Rather than reducing each requirement to a single metric, it considers multiple forms of representative evidence, summarized in Table 4, and interprets their operational meaning.

Table 4. Requirement-wise compliance snapshot.

Requirement	Representative evidence
Continuity	local persistence, recovery path, gap reduction
Governance	source identity, canonical semantics, time separation
Diagnosability	segment-wise visibility, stage discrepancy checks
Operability	modular stack, monitoring and intervention points
Reprocessability	raw-like retention, layered persistence
Evolvability	common event contract, extensible onboarding

Continuity was assessed not in terms of perfect packet arrival, but in terms of whether the operational record remained continuous and recoverable under unstable field conditions. In the implemented system, gateway reception, edge-side logging, downstream ingest, and database persistence were observable as separate stages, allowing continuity to be examined through persisted-to-expected ratios, prolonged no-data intervals, and before–after recovery patterns. Under an expected daily volume of about 48,000 events, early persistence remained near 15,870 events, and some source groups showed repeated no-data gaps of 2–5 hours. After communication-related adjustments, persisted events recovered to about 45,600 per day, while extended gap cases decreased from roughly 18 per day to fewer than two. These observations show that continuity was realized as recoverable record preservation despite intermediate instability, rather than as uninterrupted delivery.

Governance was evaluated in terms of whether heterogeneous field events were brought under a stable semantic discipline across collection, normalization, and storage. In the implemented system, source identity was governed through `source_id`, while the parser stage fixed metric definitions, timestamp interpretation, and normalization rules before downstream use. The distinction among `event_time`, `edge_ingest_time`, and `db_ingest_time` prevented ambiguity between occurrence time, pipeline reference time, and persistence time. In addition, the separation of raw-like event storage, source metadata, and derived KPI layers preserved a clear boundary between operational records and business-level abstraction. Taken together, these structures show that governance was achieved through controlled meaning, traceable identity, and stable interpretation across the pipeline.

Diagnosability was assessed in terms of whether degradation remained a black-box symptom or became visible as a discrepancy localizable to specific lifecycle segments. Because gateway-visible counts, local edge logs, parser and consumer stages, and database persistence counts could be compared, the same symptom of “too little data” could be interpreted as different failure modes depending on where the discrepancy first appeared. In some periods, gateway-visible counts were already far below expectation, directing attention to upstream collection and communication. In other cases, the divergence widened only after gateway reception, indicating the need to inspect downstream processing or storage. This segment-wise visibility was especially important in the LoRa case discussed in Section 6.2, where the main source of degradation could be narrowed to the communication and configuration layer rather than being misread as a parser or database issue.

Operability was reflected not simply in whether the system could be made to run, but in whether it could be deployed, inspected, adjusted, and maintained under limited staffing and maintenance capacity. The implemented structure separated field-side acquisition, gateway and network services, edge persistence, cloud-side stream processing, database storage, and dashboard and alert interfaces by function, reducing unnecessary coupling among acquisition, normalization, persistence, derivation, and presentation. In practice, interventions such as rejoin handling, communication-parameter adjustment, logging

verification, interval tuning, packet comparison, dashboard inspection, and alert-based response could be carried out at the relevant layer without redesigning the system as a whole. This indicates that the foundation remained operationally manageable as a field system rather than only as a one-time prototype.

Reprocessability was assessed in terms of whether collected records remained available for later reinterpretation, recalculation, correction, and recovery rather than being consumed only by the current dashboard or rule logic. The implemented system avoided transformed-only persistence by separating raw-like event storage, source metadata, and derived KPI layers, so later parser revision or KPI redefinition did not depend on reconstructing already abstracted outputs. Historical data therefore remained available for rule revision, retrospective audit, recomputation, and reconstruction of prior operational states. More importantly, the retained records were preserved in a structure that supported replay-oriented reinterpretation, which is the practical core of reprocessability in a changing manufacturing environment.

Evolvability was evaluated in terms of whether the structure could absorb new source types, process-specific measurements, monitoring functions, KPI logic, and downstream services without breaking its governance discipline. This property was reflected not only in architectural structure, but also in the system's actual expansion history. The implemented foundation accommodated heterogeneous environmental sensors, a retrofitted legacy scale, cloud-side stream processing, derived KPI generation, and dashboard and alert functions within a common event flow governed by shared identity and normalization rules. Because the structure was organized around an explicit event contract, parser-stage normalization, layered persistence, and modular responsibility separation, extensions could be handled mainly through onboarding rules, parser updates, and additional downstream consumers rather than through full pipeline redesign. This indicates that the framework supported staged expansion while preserving semantic and operational coherence.

Taken together, the evidence above indicates that the proposed framework was realized not only as a conceptual structure but also as an operational one in the field. To make this more concrete, the next subsection examines a representative field case through which the framework's practical relevance can be seen in actual operation.

6.2. Operational Case: LoRa Reliability Improvement

A field data foundation does not remain operationally reliable simply because it has once been installed. In practice, reception quality, device condition, battery state, wireless configuration, and usage patterns continue to change over time. For this reason, the value of the proposed framework lies not in whether it delivered perfect behavior from the outset, but in whether emerging failures could be detected, localized, and corrected in an operationally meaningful way. This subsection presents a representative field case in which severe reception degradation in periodic uplink collection was diagnosed and improved within the implemented foundation.

During the early operation period, plant-wide periodic collection failed to sustain its expected level. Reception across multiple monitored sensor sets dropped to roughly 33%, daily persisted events were 15,870 out of an expected 48,000, and repeated no-data intervals of 2–5 hours appeared in several monitored intervals. This was not merely a matter of lower communication quality. It directly weakened line-level visibility, reduced confidence in dashboard and alert outputs, and undermined the continuity of the operational record on which later interpretation depended.

Because the implemented system did not rely solely on final database status, the degradation could be interpreted as a segment-wise discrepancy rather than as a vague end-to-end symptom. Scheduled transmissions, gateway-visible events, edge/local logs,

ingest-stage counts, and database persistence counts could be examined together. As summarized in Table 5, the main loss had already occurred before parsing and database persistence: in a representative 24-hour window, only about 16,400 events were visible at the gateway, whereas downstream retention from gateway-visible events to database persistence remained consistently high at roughly 98–99%. This pattern indicated that the dominant loss was occurring before the parser and database layer, making the collection and communication segment the primary locus of investigation.

Table 5. Event counts across pipeline segments during reception degradation.

Stage / Metric	Expected	Observed	Ratio (%)
Scheduled transmissions	48,000	–	–
Gateway-visible events	48,000	16,400	34.2
Edge/local logged events	16,400	16,120	98.3
Ingested events	16,120	15,980	99.1
DB-persisted events	15,980	15,870	99.3

The subsequent diagnosis localized the main bottleneck not to parser failure or database loss, but to the LoRa communication-configuration layer. The problem that emerged was not a generic reduction in signal strength alone, but a network condition in which frequent periodic confirmed uplinks depended on downlink acknowledgements under limited active-channel availability. When acknowledgements were delayed or missed, retransmissions accumulated, channel contention increased, and reception became uneven across devices. In addition, stale device participation state in part of the fleet required rejoin recovery before stable collection could resume. The diagnosed issue was therefore a concrete reception bottleneck in the LoRa network path itself: ACK-dependent uplink behavior, limited effective channel use, and device-state instability were jointly degrading collection before downstream processing began.

Once the likely bottleneck had been localized, the response proceeded through targeted intervention rather than unguided trial and error. The main adjustments included reducing confirmed-mode dependence, expanding active channels, performing device rejoin, tuning transmission-related parameters, and refining monitoring intervals. After these adjustments, collection stability improved markedly. Packet reception increased from approximately 33% to 95%, daily persisted events rose from 15,870 to 45,600, extended no-data intervals decreased from about 18 cases per day to fewer than two, and average gap duration dropped from roughly 145 minutes to 18 minutes. The before–after time-series plots also show that dense missing segments visible before adjustment were substantially reduced afterward, while observation regularity and line-to-line consistency improved (Figure 8; Table 6).

The significance of this case extends beyond successful LoRa tuning. It shows, first, that R1 continuity was assessable in operational terms and that I1 durable capture could be examined through the degradation and recovery of the persisted record under severe field-side instability. Second, it shows that R3 diagnosability and I4 segment-localizable failure were realized through the comparison of scheduled transmissions, gateway-visible events, edge logs, ingest-stage records, and database persistence, which made the dominant discrepancy visible in the communication segment rather than only at the final database boundary. What this case demonstrates, therefore, is not simply that reception improved after adjustment, but that the implemented foundation made it possible to observe the failure as a structured operational discrepancy, to narrow it to a specific lifecycle segment, and to evaluate recovery within the same evidence framework. In that sense, the case

provides a concrete field illustration of the framework’s role in diagnosis and corrective improvement under constrained SME conditions.

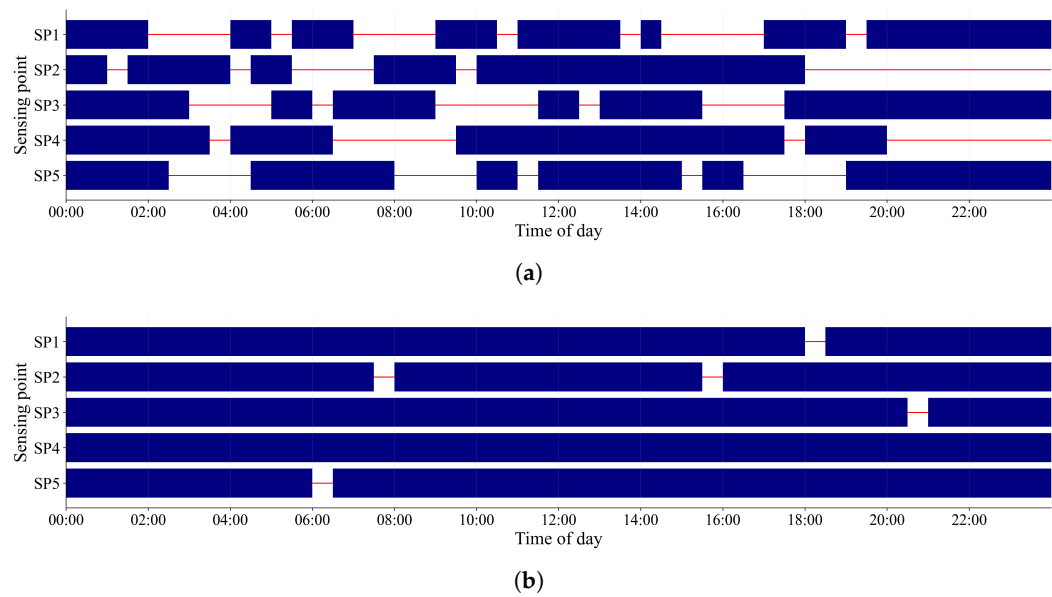


Figure 8. Reception stability across sensing points over time. (a) Before adjustment. (b) After adjustment.

Table 6. Outcome of reception reliability improvement.

Metric	Before	After	Change
Reception rate (%)	33	95	+62 pp
Persisted events/day	15,870	45,600	+187%
Average gap duration (min)	145	18	−87.6%
Long-gap events/day	18	2	−88.9%
Gateway-to-DB consistency (%)	96.8	99.4	+2.6 pp

7. Conclusion

This study argued that the main practical bottleneck of smart manufacturing in constrained SME factories lies not in downstream analytics or AI applications, but in establishing an operationally viable data foundation. In such environments, legacy equipment, fragmented systems, unstable field conditions, limited staffing, and cost sensitivity weaken the assumptions on which idealized smart-manufacturing architectures are often built. The central issue is therefore not simply how to use more data, but how to build and sustain a data foundation that remains usable under real deployment conditions.

To address this issue, this study drew on prior work on industrial data infrastructures, lifecycle connectivity, and SME digitalization, and reinterpreted recurrent SME deployment conditions as lifecycle-wide constraints and operational risks. On that basis, it proposed the *Operational Data Foundation Framework*, structured around six core requirements—continuity, governance, diagnosability, operability, reprocessability, and evolvability—and their corresponding design principles and operational invariants. The contribution is therefore not only architectural. It is also managerial, in that it provides a systematic way to specify and assess what an operational data foundation must guarantee in constrained SME factory settings.

The framework was not examined only at a conceptual level. It was implemented and operated in a real SME food-manufacturing factory. In that field setting, the study established source-side sensing, gateway and edge intake, cloud-side event processing, governed normalization, and operational dashboards and alerts as one connected foundation. The evaluation showed that the implemented system preserved controlled semantics, supported segment-wise observability, maintained replayable records, and remained manageable under constrained deployment conditions. The LoRa reception instability case further demonstrated its practical value. The problem was detected as a structured lifecycle discrepancy, localized to a specific segment, and improved through targeted intervention.

Rather than focusing on a single downstream application, this study formalized the operational data foundation that makes shop-floor data usable in practice. This matters because factories need more than data collection alone. They need records that allow operators and managers to check process conditions consistently, detect and localize abnormal situations, review past conditions when quality issues arise, and extend the system without rebuilding its entire structure. In this sense, the proposed framework offers a reference design logic for making smart manufacturing more continuous and practically operable in constrained SME settings, while also providing a basis for later extension toward more human-centric and sustainability-oriented manufacturing.

This study has several limitations. The framework was examined through a single SME food-manufacturing deployment, and its broader generalizability across multiple factories and more diverse industrial settings remains to be tested. In addition, although the implementation supported operational visibility, alerting, and machine-side intervention, the study did not directly evaluate large-scale multi-site deployment, fully automated replay workflows, or closed-loop control integration. Future work may therefore extend the framework through multi-factory replication, richer interoperability settings, automated replay-oriented pipelines, and tighter integration with analytics, digital twins, and CPPS-oriented control.

Author Contributions: For research articles with several authors, a short paragraph specifying their individual contributions must be provided. The following statements should be used “Conceptualization, X.X. and Y.Y.; methodology, X.X.; software, X.X.; validation, X.X., Y.Y. and Z.Z.; formal analysis, X.X.; investigation, X.X.; resources, X.X.; data curation, X.X.; writing—original draft preparation, X.X.; writing—review and editing, X.X.; visualization, X.X.; supervision, X.X.; project administration, X.X.; funding acquisition, Y.Y. All authors have read and agreed to the published version of the manuscript.”, please turn to the [CRediT taxonomy](#) for the term explanation. Authorship must be limited to those who have contributed substantially to the work reported.

Funding: Please add: “This research received no external funding” or “This research was funded by NAME OF FUNDER grant number XXX.” and “The APC was funded by XXX”. Check carefully that the details given are accurate and use the standard spelling of funding agency names at <https://search.crossref.org/funding>, any errors may affect your future funding.

Data Availability Statement: We encourage all authors of articles published in MDPI journals to share their research data. In this section, please provide details regarding where data supporting reported results can be found, including links to publicly archived datasets analyzed or generated during the study. Where no new data were created, or where data is unavailable due to privacy or ethical restrictions, a statement is still required. Suggested Data Availability Statements are available in section “MDPI Research Data Policies” at <https://www.mdpi.com/ethics>.

Acknowledgments: In this section you can acknowledge any support given which is not covered by the author contribution or funding sections. This may include administrative and technical support, or donations in kind (e.g., materials used for experiments). Where GenAI has been used for purposes such as generating text, data, or graphics, or for study design, data collection, analysis, or interpretation of data, please add “During the preparation of this manuscript/study, the author(s)

used [tool name, version information] for the purposes of [description of use]. The authors have reviewed and edited the output and take full responsibility for the content of this publication.”

Conflicts of Interest: The authors declare no conflicts of interest.

References

1. Rozhok, A.; Abate, R.; Manoli, E.; Nele, L. A Review of Recent Advanced Applications in Smart Manufacturing Systems. *J. Manuf. Mater. Process.* **2026**, *10*, 1. <https://doi.org/10.3390/jmmp10010001>.
2. Bueno, A.; Godinho Filho, M.; Frank, A.G. Smart production planning and control in the Industry 4.0 context: A systematic literature review. *Comput. Ind. Eng.* **2020**, *149*, 106774. <https://doi.org/10.1016/j.cie.2020.106774>.
3. Cinar, Z.M.; Nuhu, A.A.; Zeeshan, Q.; Korhan, O. Digital Twins for Industry 4.0: A Review. In *Industrial Engineering in the Digital Disruption Era*; Calisir, F.; Korhan, O., Eds.; Lecture Notes in Management and Industrial Engineering, Springer: Cham, 2020. https://doi.org/10.1007/978-3-030-42416-9_18.
4. Atalay, M.; Murat, U.; Oksuz, B.; Parlaktuna, A.M.; Pisirir, E.; Testik, M.C. Digital twins in manufacturing: systematic literature review for physical–digital layer categorization and future research directions. *Int. J. Comput. Integr. Manuf.* **2022**, *35*, 679–705. <https://doi.org/10.1080/0951192X.2021.2022762>.
5. Song, Y. A Comprehensive Review of Key Technologies and Applications in Smart Manufacturing Systems: From Digital Foundations to Intelligent Applications. In Proceedings of the Proceedings of the 2025 2nd International Conference on Industrial Automation and Robotics, New York, NY, USA, 2025; IAR '25, pp. 328–333. <https://doi.org/10.1145/3778886.3778938>.
6. Nguyen, P.; Kim, M.; Nichols, E.; Yoon, H.S. AI-Driven Digital Twins for Manufacturing: A Review Across Hierarchical Manufacturing System Levels. *Sensors* **2025**, *26*, 124. <https://doi.org/10.3390/s26010124>.
7. Tao, F.; Zhang, M. Digital Twin Shop-Floor: A New Shop-Floor Paradigm Towards Smart Manufacturing. *IEEE Access* **2017**, *5*, 20418–20427. <https://doi.org/10.1109/ACCESS.2017.2756069>.
8. Ding, K.; Chan, F.T.S.; Zhang, X.; Zhou, G.; Zhang, F. Defining a Digital Twin-based Cyber-Physical Production System for autonomous manufacturing in smart shop floors. *Int. J. Prod. Res.* **2019**, *57*, 6315–6334. <https://doi.org/10.1080/00207543.2019.1566661>.
9. Ojstersek, R.; Javernik, A.; Buchmeister, B. Optimizing smart manufacturing systems using digital twin. *Adv. Prod. Eng. Manag.* **2023**, *18*, 475–485. <https://doi.org/10.14743/apem2023.4.486>.
10. Qamsane, Y.; Chen, C.Y.; Balta, E.C.; Kao, B.C.; Mohan, S.; Moyne, J.; Tilbury, D.; Barton, K. A Unified Digital Twin Framework for Real-time Monitoring and Evaluation of Smart Manufacturing Systems. In Proceedings of the 2019 IEEE 15th International Conference on Automation Science and Engineering (CASE), 2019, pp. 1394–1401. <https://doi.org/10.1109/COASE.2019.8843269>.
11. Ciano, M.P.; Pozzi, R.; Rossi, T.; Strozzi, F. Digital twin-enabled smart industrial systems: a bibliometric review. *Int. J. Comput. Integr. Manuf.* **2021**, *34*, 690–708. <https://doi.org/10.1080/0951192X.2020.1852600>.
12. Ismail, A.; Truong, H.L.; Kastner, W. Manufacturing process data analysis pipelines: a requirements analysis and survey. *J. Big Data* **2019**, *6*, 1. <https://doi.org/10.1186/s40537-018-0162-3>.
13. Raptis, T.P.; Passarella, A.; Conti, M. Data Management in Industry 4.0: State of the Art and Open Challenges. *IEEE Access* **2019**, *7*, 97052–97093. <https://doi.org/10.1109/ACCESS.2019.2929296>.
14. Saqlain, M.; Piao, M.; Shim, Y.; Lee, J.Y. Framework of an IoT-based Industrial Data Management for Smart Manufacturing. *J. Sens. Actuator Netw.* **2019**, *8*, 25. <https://doi.org/10.3390/jsan8020025>.
15. O'Donovan, P.; Leahy, K.; Bruton, K.; O'Sullivan, D.T.J. An industrial big data pipeline for data-driven analytics maintenance applications in large-scale smart manufacturing facilities. *J. Big Data* **2015**, *2*, 25. <https://doi.org/10.1186/s40537-015-0034-z>.
16. Zeid, A.; Sundaram, S.; Moghaddam, M.; Kamarthi, S.; Marion, T. Interoperability in Smart Manufacturing: Research Challenges. *Machines* **2019**, *7*, 21. <https://doi.org/10.3390/machines7020021>.
17. Pereira, R.M.; Szejka, A.L.; Canciglieri Junior, O. Towards an information semantic interoperability in smart manufacturing systems: contributions, limitations and applications. *Int. J. Comput. Integr. Manuf.* **2021**, *34*, 422–439. <https://doi.org/10.1080/0951192X.2021.1891571>.
18. Modoni, G.E.; Doukas, M.; Terkaj, W.; Sacco, M.; Mourtzis, D. Enhancing factory data integration through the development of an ontology: from the reference models reuse to the semantic conversion of the legacy models. *Int. J. Comput. Integr. Manuf.* **2017**, *30*, 1043–1059. <https://doi.org/10.1080/0951192X.2016.1268720>.
19. Hildebrandt, C.; Köcher, A.; Küstner, C.; López-Enríquez, C.M.; Müller, A.W.; Caesar, B.; Gundlach, C.S.; Fay, A. Ontology Building for Cyber–Physical Systems: Application in the Manufacturing Domain. *IEEE Trans. Autom. Sci. Eng.* **2020**, *17*, 1266–1282. <https://doi.org/10.1109/TASE.2020.2991777>.
20. Morris, K.C.; Lu, Y.; Frechette, S. Foundations of Information Governance for Smart Manufacturing. *Smart Sustain. Manuf. Syst.* **2020**, *4*, 43–61. <https://doi.org/10.1520/SSMS20190041>.

21. Westermann, T.; Hranisavljevic, N.; Fay, A. Accessing and Interpreting OPC UA Event Traces based on Semantic Process Descriptions. In Proceedings of the 2022 IEEE 27th International Conference on Emerging Technologies and Factory Automation (ETFA), Stuttgart, Germany, 2022; pp. 1–7. <https://doi.org/10.1109/ETFA52439.2022.9921565>. 983–984–985
22. Ren, S.; Zhang, Y.; Liu, Y.; Sakao, T.; Huisingh, D.; Almeida, C.M.V.B. A comprehensive review of big data analytics throughout product lifecycle to support sustainable smart manufacturing: A framework, challenges and future research directions. *J. Clean. Prod.* **2019**, *210*, 1343–1365. <https://doi.org/10.1016/j.jclepro.2018.11.025>. 986–987–988
23. Tao, F.; Qi, Q.; Liu, A.; Kusiak, A. Data-driven smart manufacturing. *J. Manuf. Syst.* **2018**, *48*, 157–169. <https://doi.org/10.1016/j.jmsy.2018.01.006>. 989–990
24. Bernstein, W.Z.; Hedberg Jr., T.D.; Helu, M.; Feeney, A.B. Contextualising manufacturing data for lifecycle decision-making. *Int. J. Prod. Lifecycle Manag.* **2018**, *10*, 326–347. <https://doi.org/10.1504/IJPLM.2017.090328>. 991–992
25. Hedberg Jr., T.D.; Feeney, A.B.; Helu, M.; Camelio, J.A. Toward a Lifecycle Information Framework and Technology in Manufacturing. *J. Comput. Inf. Sci. Eng.* **2017**, *17*, 021010. <https://doi.org/10.1115/1.4034132>. 993–994
26. Abdel-Aty, T.A.; Negri, E. Conceptualizing the digital thread for smart manufacturing: a systematic literature review. *J. Intell. Manuf.* **2024**, *35*, 3629–3653. <https://doi.org/10.1007/s10845-024-02407-1>. 995–996
27. Zhang, Q.; Liu, J.; Chen, X. A Literature Review of the Digital Thread: Definition, Key Technologies, and Applications. *Systems* **2024**, *12*, 70. <https://doi.org/10.3390/systems12030070>. 997–998
28. Bianchini, D.; Fapanni, T.; Garda, M.; Leotta, F.; Mecella, M.; Rula, A.; Sardini, E. Digital Thread for Smart Products: A Survey on Technologies, Challenges, and Opportunities in Service-Oriented Supply Chains. *IEEE Access* **2024**, *12*, 125284–125305. <https://doi.org/10.1109/ACCESS.2024.3454375>. 999–1000–1001
29. Hedberg Jr., T.D.; Bajaj, M.; Camelio, J.A. Using Graphs to Link Data Across the Product Lifecycle for Enabling Smart Manufacturing Digital Threads. *J. Comput. Inf. Sci. Eng.* **2020**, *20*, 011011. <https://doi.org/10.1115/1.4044921>. 1002–1003
30. Kwon, S.; Monnier, L.V.; Barbau, R.; Bernstein, W.Z. Enriching standards-based digital thread by fusing as-designed and as-inspected data using knowledge graphs. *Adv. Eng. Inform.* **2020**, *46*, 101102. <https://doi.org/10.1016/j.aei.2020.101102>. 1004–1005
31. Schmidt, N.; Lueder, A. The Flow and Reuse of Data: Capabilities of AutomationML in the Production System Life Cycle. *IEEE Ind. Electron. Mag.* **2018**, *12*, 59–63. <https://doi.org/10.1109/MIE.2018.2818748>. 1006–1007
32. Monnier, L.V.; Shao, G.; Foufou, S. A Methodology for Digital Twins of Product Lifecycle Supported by Digital Thread. In Proceedings of the Proceedings of the ASME 2022 International Mechanical Engineering Congress and Exposition, Columbus, Ohio, USA, 2022; p. V02BT02A023. <https://doi.org/10.1115/IMECE2022-95182>. 1008–1009–1010
33. Mittal, S.; Khan, M.A.; Romero, D.; Wuest, T. A critical review of smart manufacturing & Industry 4.0 maturity models: Implications for small and medium-sized enterprises (SMEs). *J. Manuf. Syst.* **2018**, *49*, 194–214. <https://doi.org/10.1016/j.jmsy.2018.10.005>. 1011–1012–1013
34. Mittal, S.; Khan, M.A.; Purohit, J.K.; Menon, K.; Romero, D.; Wuest, T. A smart manufacturing adoption framework for SMEs. *Int. J. Prod. Res.* **2020**, *58*, 1555–1573. <https://doi.org/10.1080/00207543.2019.1661540>. 1014–1015
35. Ghobakhloo, M.; Ching, N.T. Adoption of digital technologies of smart manufacturing in SMEs. *J. Ind. Inf. Integr.* **2019**, *16*, 100107. <https://doi.org/10.1016/j.jii.2019.100107>. 1016–1017
36. Krishnan, R. Challenges and benefits for small and medium enterprises in the transformation to smart manufacturing: a systematic literature review and framework. *J. Manuf. Technol. Manag.* **2024**, *35*, 918–938. <https://doi.org/10.1108/JMTM-07-2022-0255>. 1018–1019
37. Alqoud, A.; Schaefer, D.; Milisavljevic-Syed, J. Industry 4.0: a systematic review of legacy manufacturing system digital retrofitting. *Manufacturing Rev.* **2022**, *9*, 32. https://doi.org/10.1007/978-3-030-57997-5_13. 1020–1021
38. Etz, D.; Brantner, H.; Kastner, W. Smart Manufacturing Retrofit for Brownfield Systems. *Procedia Manuf.* **2020**, *42*, 327–332. <https://doi.org/10.1016/j.promfg.2020.02.085>. 1022–1023
39. Tran, T.A.; Ruppert, T.; Eigner, G.; Abonyi, J. Retrofitting-Based Development of Brownfield Industry 4.0 and Industry 5.0 Solutions. *IEEE Access* **2022**, *10*, 64348–64374. <https://doi.org/10.1109/ACCESS.2022.3182491>. 1024–1025
40. Park, H.M.; Jeon, J.W. OPC UA based Universal Edge Gateway for Legacy Equipment. In Proceedings of the 2019 IEEE 17th International Conference on Industrial Informatics (INDIN), Helsinki, Finland, 2019; pp. 1002–1007. <https://doi.org/10.1109/INDIN41052.2019.8972187>. 1026–1027–1028
41. Huang, D.; Chin, C.P.Y. The Barriers and Challenges in Smart Manufacturing Adoption for SMEs: A Review. *J. Adv. Manuf. Syst.* **2025**, *24*, 533–557. <https://doi.org/10.1142/S0219686725500246>. 1029–1030
42. Narwane, V.S.; Raut, R.D.; Gardas, B.B.; Narkhede, B.E.; Awasthi, A. Examining smart manufacturing challenges in the context of micro, small and medium enterprises. *Int. J. Comput. Integr. Manuf.* **2022**, *35*, 1395–1412. <https://doi.org/10.1080/0951192X.2022.2078508>. 1031–1032–1033
43. Kumar, R.; Dutta, G.; Phanden, R.K. Digitalization Adoption Barriers in the Context of Sustainability and Operational Excellence: Implications for SMEs. *Eng. Manag. J.* **2025**, *37*, 355–371. <https://doi.org/10.1080/10429247.2024.2372519>. 1034–1035
44. Hulla, M.; Herstätter, P.; Wolf, M.; Ramsauer, C. Towards digitalization in production in SMEs – A qualitative study of challenges, competencies and requirements for trainings. *Procedia CIRP* **2021**, *104*, 887–892. <https://doi.org/10.1016/j.procir.2021.11.149>. 1036–1037

45. Chavez, Z.; Baalsrud Hauge, J.; Bellgran, M. A Conceptual Model for Deploying Digitalization in SMEs Through Capability Building. In *Advances in Production Management Systems. Towards Smart and Digital Manufacturing*; Lalic, B.; Majstorovic, V.; Marjanovic, U.; von Cieminski, G.; Romero, D., Eds.; Springer: Cham, 2020; Vol. 592, *IFIP Advances in Information and Communication Technology*. https://doi.org/10.1007/978-3-030-57997-5_13. 1038–1041
46. Banerjee, A.; Jayaraman, P.P.; Fizza, K.; Wang, S.; Jin, J.; Ghaderi, H. Low-cost digital manufacturing solution for process manufacturing SMEs - Lesson and experiences from real-world pilot. *IET Conf. Proc.* **2024**, *2024*, 109–115. <https://doi.org/10.1049/icp.2024.3494>. 1042–1044
47. Foidl, H.; Golendukhina, V.; Ramler, R.; Felderer, M. Data pipeline quality: Influencing factors, root causes of data-related issues, and processing problem areas for developers. *J. Syst. Softw.* **2024**, *207*, 111855. <https://doi.org/10.1016/j.jss.2023.111855>. 1045–1046
48. Rauch, E.; Dallasega, P.; Unterhofer, M. Requirements and Barriers for Introducing Smart Manufacturing in Small and Medium-Sized Enterprises. *IEEE Eng. Manag. Rev.* **2019**, *47*, 87–94. <https://doi.org/10.1109/EMR.2019.2931564>. 1047–1048
49. Gao, C.; Wang, Z.; Chen, Y. On the Connectivity of Highly Dynamic Wireless Sensor Networks in Smart Factory. In *Proceedings of the 2019 International Conference on Networking and Network Applications (NaNA)*, Daegu, Korea (South), 2019; pp. 208–212. <https://doi.org/10.1109/NaNA.2019.00045>. 1049–1051
50. Noor-A-Rahim, M.; John, J.; Firyaguna, F.; Sherazi, H.H.R.; Kushch, S.; Vijayan, A.; O'Connell, E.; Pesch, D.; O'Flynn, B.; O'Brien, W.; et al. Wireless Communications for Smart Manufacturing and Industrial IoT: Existing Technologies, 5G and Beyond. *Sensors* **2023**, *23*, 73. <https://doi.org/10.3390/s23010073>. 1052–1054
51. Hawkrigde, G.; Hernandez, M.P.; de Silva, L.; Terrazas, G.; Tlegenov, Y.; McFarlane, D.; Thorne, A. Tying Together Solutions for Digital Manufacturing: Assessment of Connectivity Technologies & Approaches. In *Proceedings of the 2019 24th IEEE International Conference on Emerging Technologies and Factory Automation (ETFA)*, Zaragoza, Spain, 2019; pp. 1383–1387. <https://doi.org/10.1109/ETFA.2019.8869411>. 1055–1058
52. Nagorny, K.; Scholze, S.; Colombo, A.W.; Oliveira, J.B. A DIN Spec 91345 RAMI 4.0 Compliant Data Pipelining Model: An Approach to Support Data Understanding and Data Acquisition in Smart Manufacturing Environments. *IEEE Access* **2020**, *8*, 223114–223129. <https://doi.org/10.1109/ACCESS.2020.3045111>. 1059–1061
53. Nasirinejad, M.; Afshari, H.; Sampalli, S. Challenges and Solutions to Adopt Smart Maintenance in SMEs: A Literature Review and Research Agenda. *IFAC-PapersOnLine* **2024**, *58*, 917–922. <https://doi.org/10.1016/j.ifacol.2024.09.164>. 1062–1063
54. Doyle, F.; Cosgrove, J. Steps towards digitization of manufacturing in an SME environment. *Procedia Manuf.* **2019**, *38*, 540–547. <https://doi.org/10.1016/j.promfg.2020.01.068>. 1064–1065
55. McFarlane, D.; Ratchev, S.; de Silva, L.; Hawkrigde, G.; Schönfuß, B.; Terrazas Angulo, G. Digitalisation for SME Manufacturers: A Framework and a Low-Cost Approach. *IFAC-PapersOnLine* **2022**, *55*, 414–419. <https://doi.org/10.1016/j.ifacol.2022.04.229>. 1066–1067

Disclaimer/Publisher's Note: The statements, opinions and data contained in all publications are solely those of the individual author(s) and contributor(s) and not of MDPI and/or the editor(s). MDPI and/or the editor(s) disclaim responsibility for any injury to people or property resulting from any ideas, methods, instructions or products referred to in the content. 1068–1070